

Matplotlib

- 시각화 패키지
- 파이썬 표준 시각화 도구
- 2D 평면 그래프에 관한 다양한 포맷과 기능 지원
- 데이터 분석 결과를 시각화 하는데 필요한 다양한 기능 제공
- <https://matplotlib.org/>

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

1. 패키지 사용법

- matplotlib 주 패키지 사용시
 - import matplotlib as mpl
- pyplot 서브 패키지 사용시 : 주로 사용
 - import matplotlib.pyplot as plt

```
In [2]: import matplotlib
import matplotlib.pyplot as plt
matplotlib.__version__
```

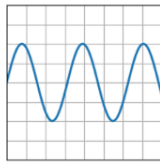
Out[2]: '3.11.1'

2. 지원 되는 플롯 유형

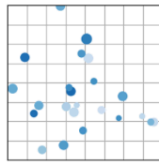
- 선그래프(line plot) : plot()
- 막대그래프(bar chart) : bar(), barh()
- 산점도(scatter plot) : scatter()
- 히스토그램(histogram) : hist()
- 상자그림표(box plot) : boxplot()
- 원그래프(pie chart) : pie()
- 기타 다양한 유형의 차트/플롯을 지원
- https://matplotlib.org/stable/plot_types/index.html

Basic

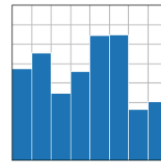
Basic plot types, usually y versus x.



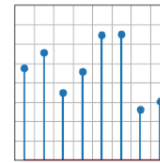
`plot(x, y)`



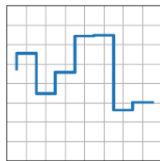
`scatter(x, y)`



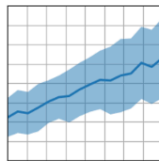
`bar(x, height) / barh(y, width)`



`stem(x, y)`



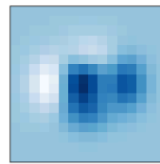
`step(x, y)`



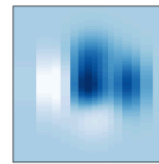
`fill_between(x, y1, y2)`

Plots of arrays and fields

Plotting for arrays of data $z(x, y)$ and fields $u(x, y)$, $v(x, y)$.



`imshow(Z)`



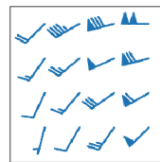
`pcolormesh(X, Y, Z)`



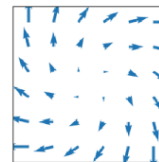
`contour(X, Y, Z)`



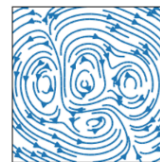
`contourf(X, Y, Z)`



`barbs(X, Y, U, V)`



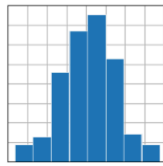
`quiver(X, Y, U, V)`



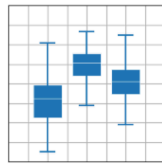
`streamplot(X, Y, U, V)`

Statistics plots

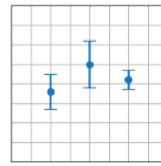
Plots for statistical analysis.



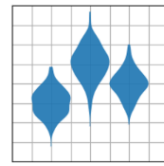
hist(x)



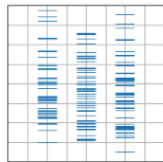
boxplot(X)



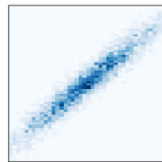
errorbar(x, y, yerr, xerr)



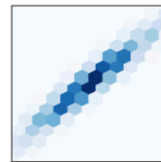
violinplot(D)



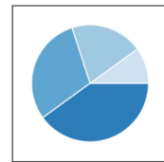
eventplot(D)



hist2d(x, y)



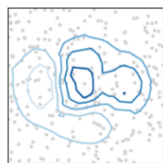
hexbin(x, y, C)



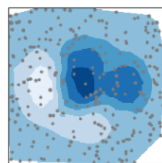
pie(x)

Unstructured coordinates

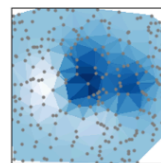
Sometimes we collect data z at coordinates (x, y) and want to visualize as a contour. Instead of gridding the data and then using `contour`, we can use a triangulation algorithm and fill the triangles.



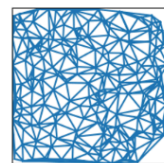
tricontour(x, y, z)



tricontourf(x, y, z)

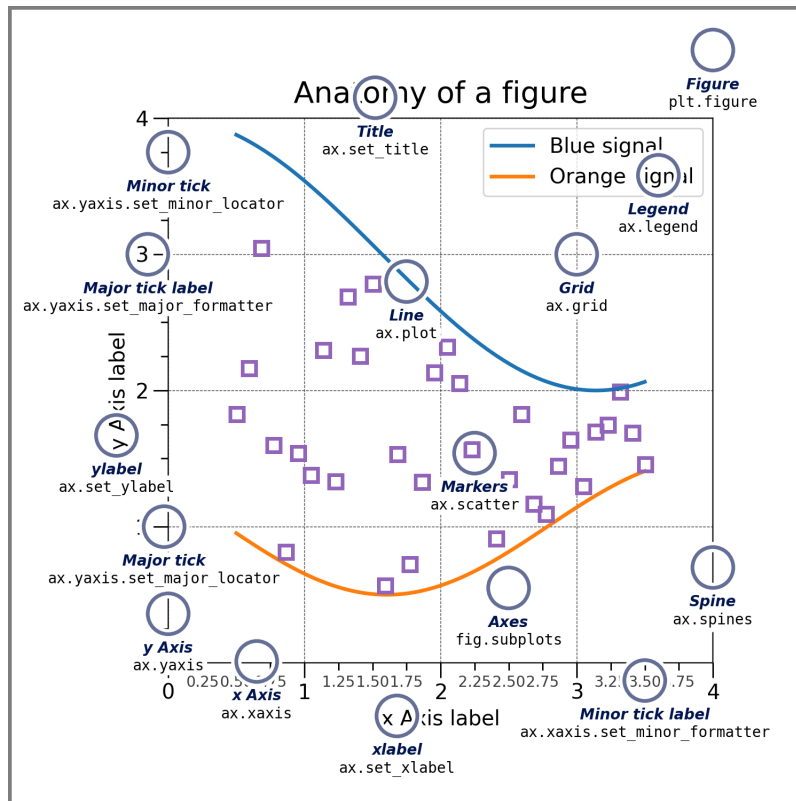


tripcolor(x, y, z)



triplot(x, y)

3. Matplotlib Figure의 구성 요소



https://matplotlib.org/stable/tutorials/introductory/quick_start.html#sphx-glr-tutorials-introductory-quick-start-py

1) pyplot 관련 함수 및 속성(시각화 옵션들)

https://matplotlib.org/stable/api/pyplot_summary.html

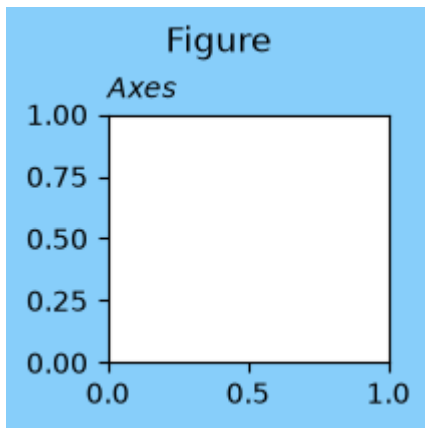
- `figure(x,y)` : 그래프 크기 설정 : 단위 인치
- `title()` : 제목 출력
- `xlabel()` : x축라벨(값)
- `ylabel()` : y축라벨(값)
- `xlim()` : x 축 범위
- `ylim()` : y 축 범위
- `xticks(), yticks()` : 축과 값을 잇는 실선
- `legend()` : 범례
- `grid()` : 그래프 배경으로 grid 사용 결정 함수

2) 플롯 영역(figure) 생성, 표출, 저장

<https://matplotlib.org/stable/users/explain/figures.html>

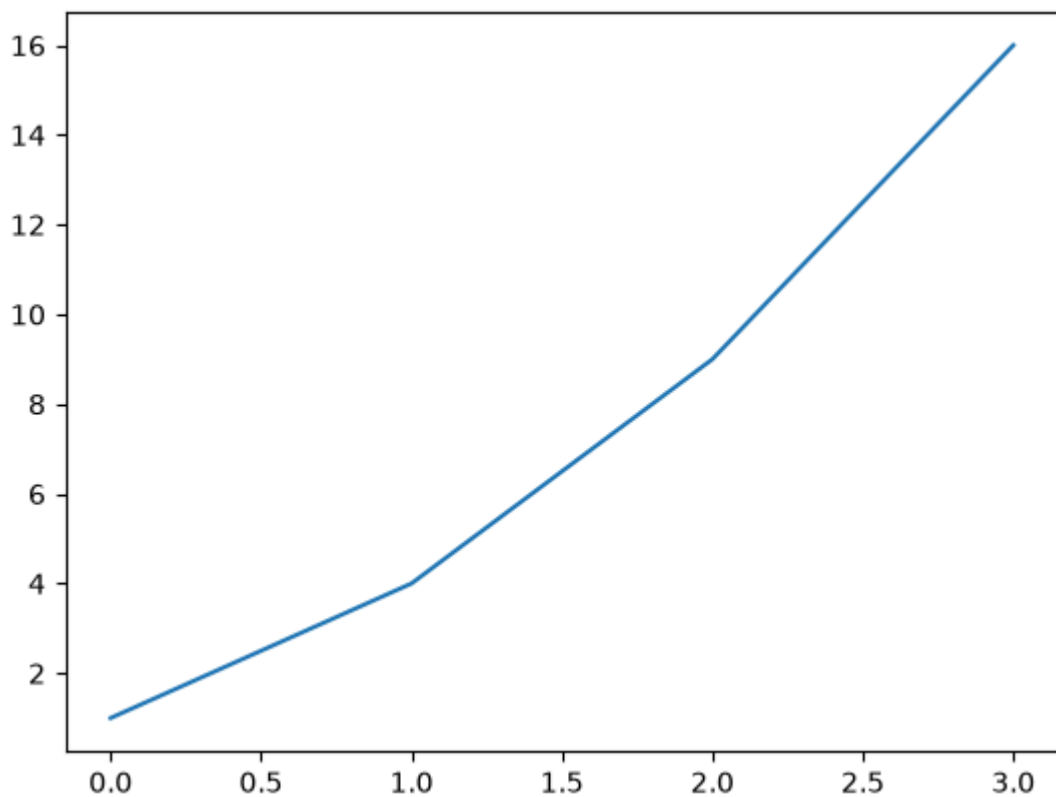
```
In [4]: fig = plt.figure(figsize=(2,2), facecolor='lightskyblue', layout='constrained')
fig.suptitle('Figure')

ax = fig.add_subplot()
ax.set_title('Axes', loc='left', fontstyle='oblique', fontsize='medium')
plt.show()
```



기본 plot 그리기

```
In [5]: plt.plot([1,4,9,16]) # x축의 값이 [0,1,2,3]으로 자동 생성
plt.show()
```



plt.show()

- 시각화 명령(그래프 그리는 함수) 후 실제 차트를 렌더링하고 마우스 이벤트 등의 지시를 기다리는 함수
- 주피터 노트북에서는 셀 단위로 플롯 명령을 자동으로 렌더링해주므로 show 명령이 필요없지만,
- 일반 파이썬 인터프리터로 가동되는 환경에서 항상 마지막에 실행하도록 코드를 작성함

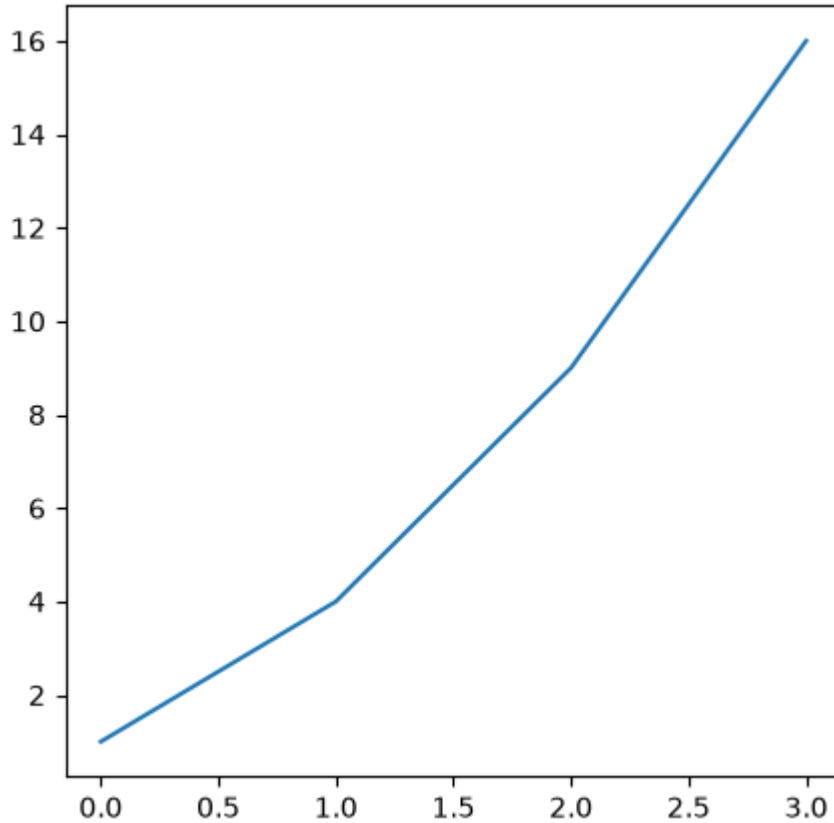
3) 플롯의 영역(figure) 크기 변경 : figure(figsize=(5,5))

- 단위 : 인치(가로,세로)

- 기본크기는 default: (6.4, 4.8)

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.figure.html

```
In [6]: plt.figure(figsize=(5,5))
plt.plot([1,4,9,16])
plt.show()
```



4) 플롯 제목 : title()

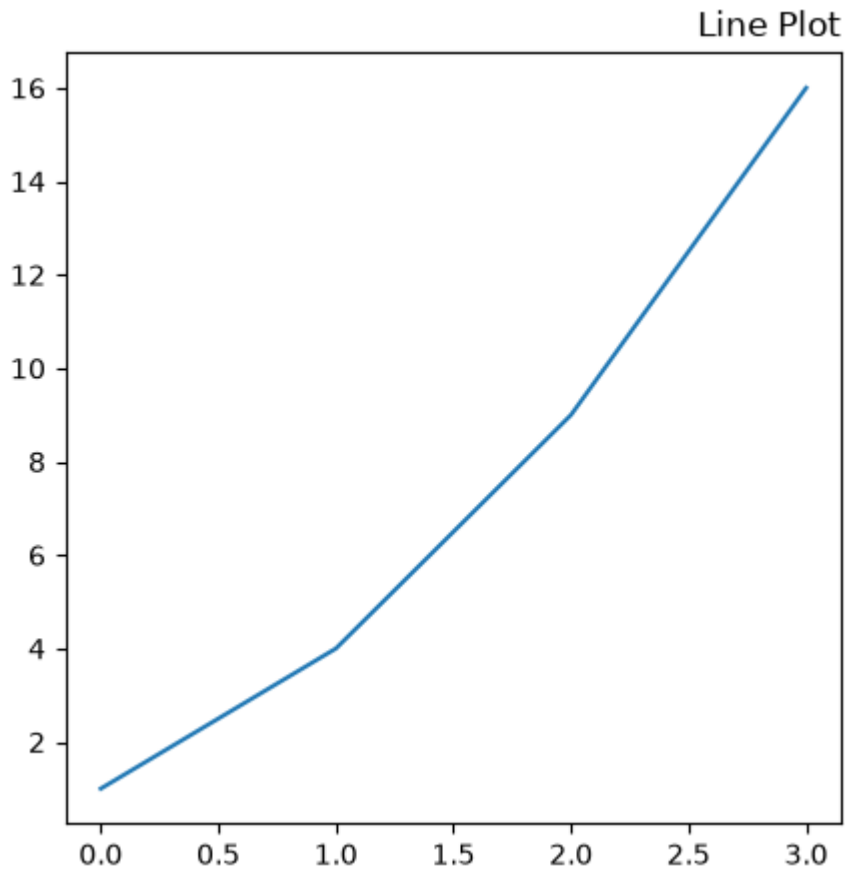
- 그래프 제목 설정
- `pyplot.title(data, loc=, pad=, fontdict=, fontsize=)`
 - `loc= 'right'|'left'| 'center'| 'right'`로 설정할 수 있으며 디폴트는 'center'
 - `pad=point` 은 타이틀과 그래프와의 간격 (오프셋)을 포인트(숫자) 단위로 설정
 - `fontdict` : 폰트크기, 서체 등의 요소를 딕셔너리로 지정
 - `fontsize=제목폰트크기`

```
fontdict = {'fontsize': rcParams['axes.titlesize'],
            'fontweight': rcParams['axes.titleweight'],
            'color': rcParams['axes.titlecolor'],
            'verticalalignment': 'baseline',
            'horizontalalignment': loc}
```

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.title.html

```
In [7]: plt.figure(figsize=(5,5))
plt.plot([1,4,9,16])
```

```
plt.title('Line Plot', loc='right')
plt.show()
```



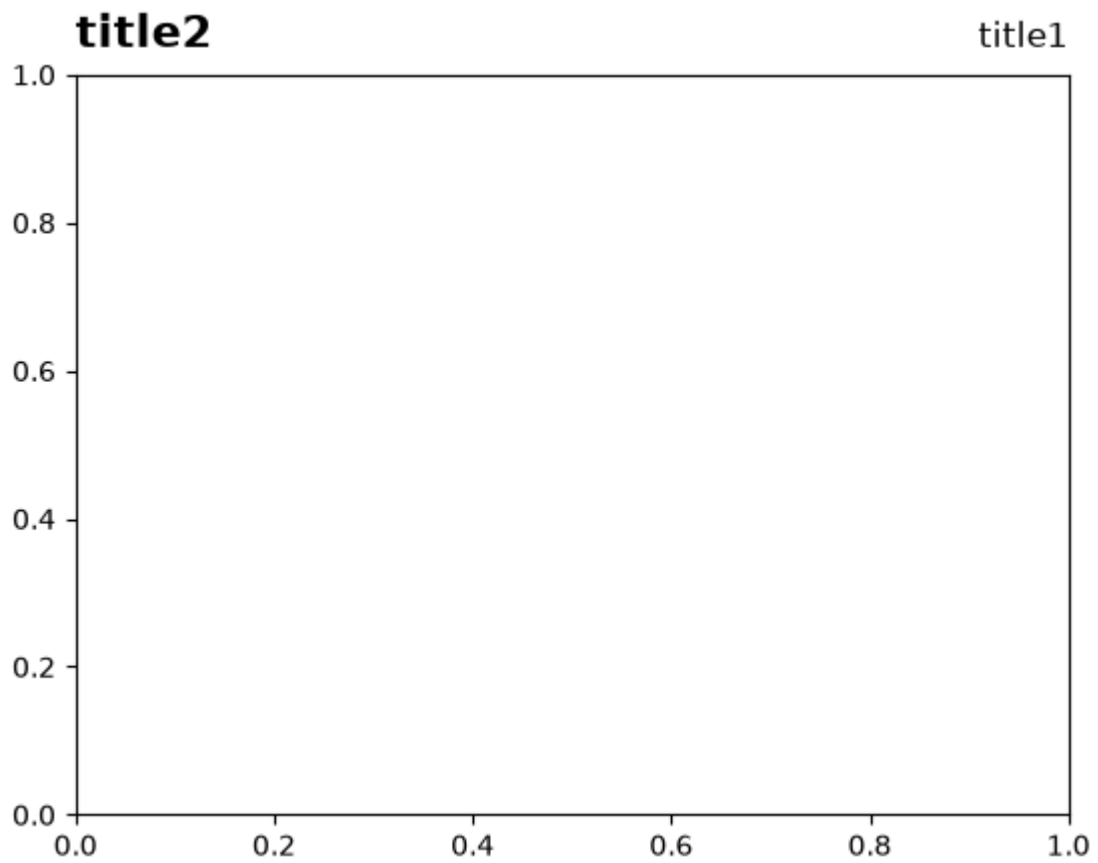
5) 플롯 Title 폰트 관련 지정

- 딕셔너리형식으로 fontsize 및 fontweight 등 지정 가능
- 한글로 제목, 축제목 등을 표시하려는 경우 폰트를 지정해야 함

```
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['font.family'] = 'Malgun Gothic' # Windows인 경우
'NanumGothic'
# plt.rcParams['font.family'] = 'AppleGothic' # MacOS인 경우
```

```
import platform
plat = platform.system()
if plat == 'Windows':
    plt.rcParams['font.family'] = 'Malgun Gothic'
elif plat == 'Darwin':
    plt.rcParams['font.family'] = 'AppleGothic'
else:
    plt.rcParams['font.family'] = 'NanumGothic'
```

```
In [8]: plt.title('title1', loc='right', pad=20)
title_font = {
    'fontsize':16, 'fontweight':'bold'
}
plt.title('title2', fontdict=title_font, loc='left', pad=10)
plt.show()
```



- 한글을 표시하기 위해서는 한글폰트 설정이 필요

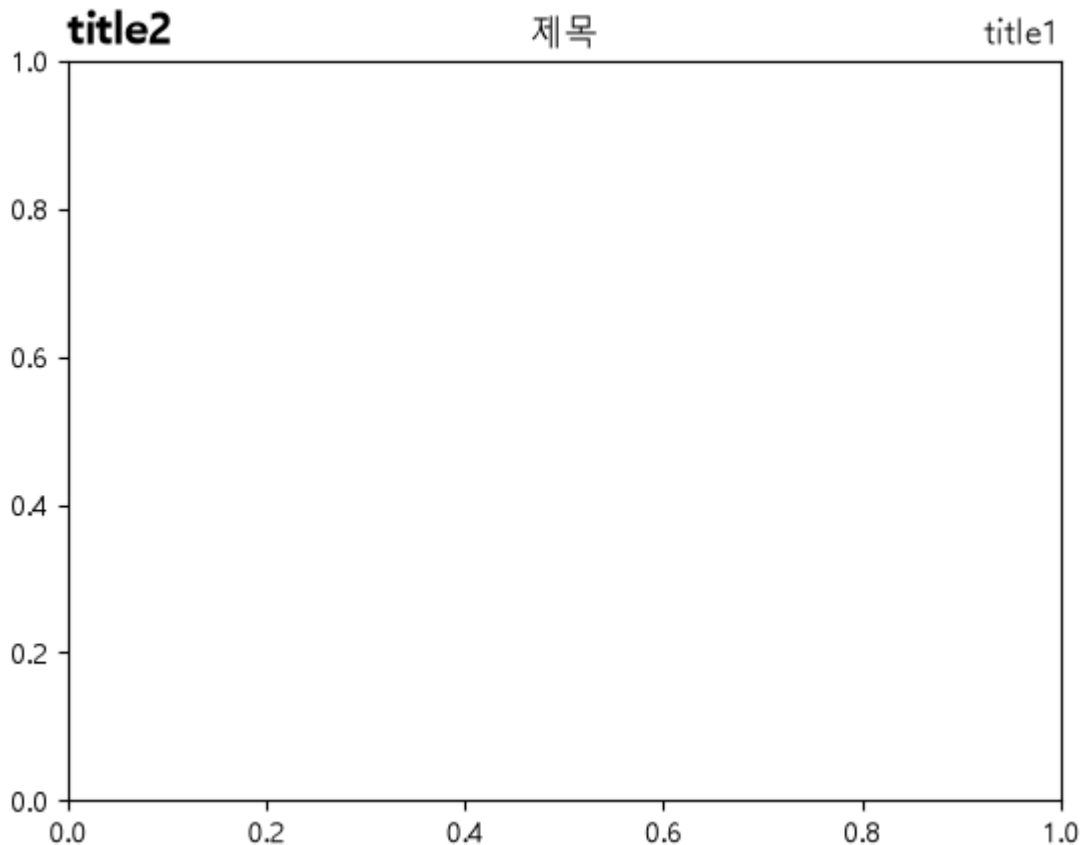
```
In [10]: # 설치된 폰트 목록 확인
import matplotlib.font_manager as fm

for font in fm.fontManager.ttflist:
    if 'Gothic' in font.name:
        print(font.name)
```

Franklin Gothic Demi
NanumBarunGothic
Malgun Gothic
Copperplate Gothic Light
Yu Gothic
Yu Gothic Medium
Yu Gothic UI
Century Gothic
Malgun Gothic
Yu Gothic
Yu Gothic UI
Yu Gothic UI
Yu Gothic UI Semibold
NanumGothic
Franklin Gothic Medium Cond
NanumGothic
NanumGothic Light
Franklin Gothic Medium
Franklin Gothic Demi
NanumBarunGothic
NanumBarunGothic UltraLight
Malgun Gothic
Malgun Gothic Semilight
NanumBarunGothic
NanumBarunGothic Light
NanumGothic
Franklin Gothic Book
HYGothic-Extra
Franklin Gothic Medium
Copperplate Gothic Bold
NanumGothic
NanumGothicExtraBold
HYGothic-Medium
Yu Gothic
Yu Gothic Light
Yu Gothic UI
Yu Gothic UI Light
Hancom Gothic
Century Gothic
Yu Gothic
Yu Gothic UI
Yu Gothic UI Semilight
Franklin Gothic Heavy
MS Gothic
MS UI Gothic
MS PGothic
Franklin Gothic Demi Cond
Hancom Gothic
Franklin Gothic Book
Century Gothic
NanumBarunGothic
Franklin Gothic Heavy
Showcard Gothic
Century Gothic

```
In [11]: # 한글폰트 설정
plt.rcParams['axes.unicode_minus'] = False
# 음수(-) 기호가 깨지는 것 방지 (유니코드 마이너스 기호 사용하지 않음)
plt.rcParams['font.family'] = 'Malgun Gothic' # Windows인 경우 'NanumGothic'
# plt.rcParams['font.family'] = 'AppleGothic' # MacOS인 경우
```

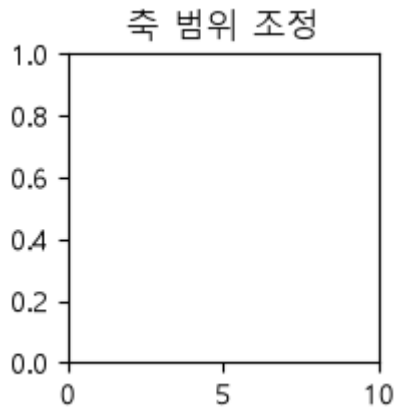
```
In [12]: plt.title('title1', loc='right', pad=20)
title_font = {
    'fontsize':16, 'fontweight':'bold'
}
plt.title('title2', fontdict=title_font, loc='left', pad=10)
plt.title('제목')
plt.show()
```



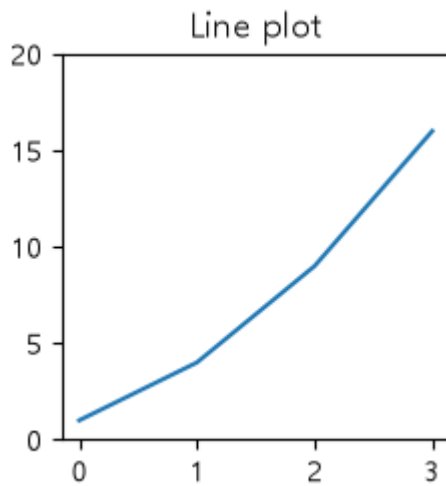
6) 플롯 축 범위 조정 : xlim(), ylim()

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xlim.html
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.ylim.html

```
In [14]: plt.figure(figsize=(2,2))
plt.title('축 범위 조정')
plt.xlim(0,10) # 기본 축범위는 (0,1)
plt.show()
```



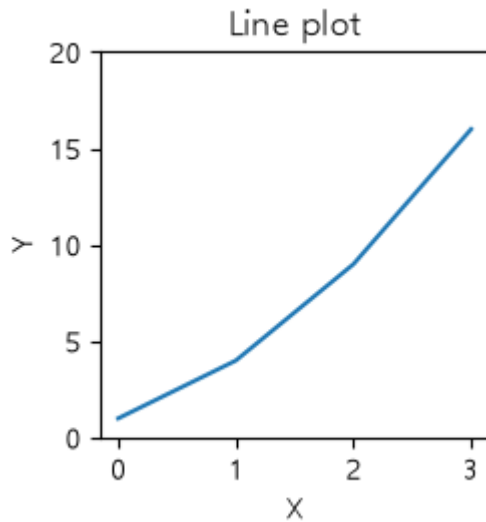
```
In [17]: plt.figure(figsize=(2.5,2.5))
plt.plot([1,4,9,16])
plt.title('Line plot')
plt.ylim(0,20)
plt.show()
```



7) 플롯의 축라벨 : xlabel(), ylabel()

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xlabel.html
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.ylabel.html

```
In [18]: plt.figure(figsize=(2.5,2.5))
plt.plot([1,4,9,16])
plt.title('Line plot')
plt.ylim(0,20)
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



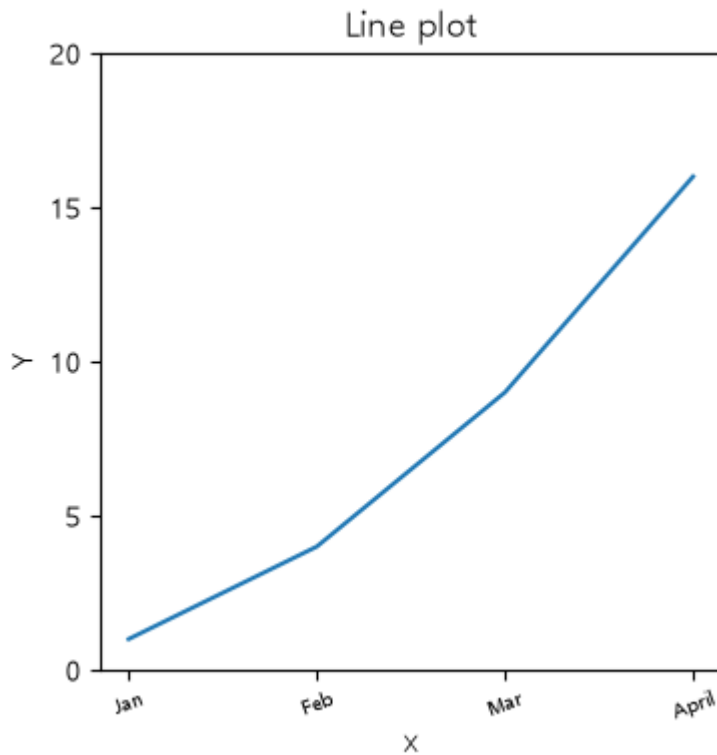
8) 플롯 축의 눈금 설정 : xticks(), yticks()

- tick은 축상의 위치 표시 지점
- 축에 간격을 구분하기 위해 표시하는 눈금
- xticks([x축값1,x축값2,...]) #튜플,리스트등 이용해서 축 값(위치 나열)
- yticks([y축값1,y축값2,...]) #튜플,리스트등 이용해서 축 값(위치 나열)
- tick_params()
- tick label(눈금 레이블) : tick에 써진 숫자 혹은 글자
- 함수형식:

```
matplotlib.pyplot.xticks(ticks=None, labels=None, *,
minor=False, **kwargs)
```

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xticks.html
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.yticks.html

```
In [27]: plt.figure(figsize=(4,4))
plt.plot([1,4,9,16])
plt.title('Line plot')
plt.ylim(0,20)
plt.xlabel('X', fontdict={'fontsize':8})
plt.ylabel('Y')
plt.xticks(ticks=[0,1,2,3], labels=['Jan','Feb','Mar','April'],
rotation=20, fontdict={'fontsize':7})
plt.yticks([0,5,10,15,20])
plt.show()
```



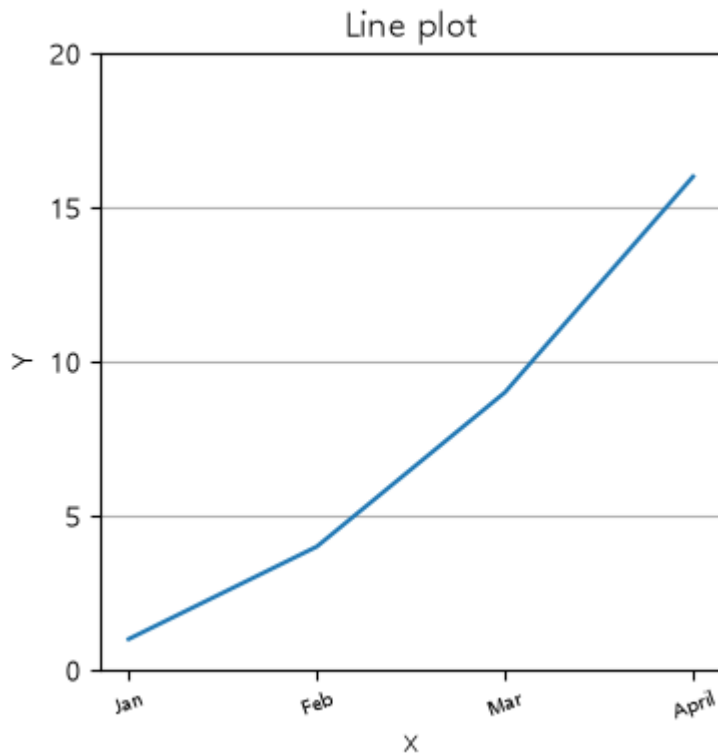
9) 플롯의 그리드 설정 : grid()

- 함수형식 :

```
matplotlib.pyplot.grid(visible=None, which='major', axis='both' |  
x | y', **kwargs)
```

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.grid.html

```
In [31]: plt.figure(figsize=(4,4))  
plt.plot([1,4,9,16])  
plt.title('Line plot')  
plt.ylim(0,20)  
plt.xlabel('X', fontdict={'fontsize':8})  
plt.ylabel('Y')  
plt.xticks(ticks=[0,1,2,3], labels=['Jan','Feb','Mar','April'],  
           rotation=20, fontdict={'fontsize':7})  
plt.yticks([0,5,10,15,20])  
plt.grid(axis='y') # axis='both', 'x', 'y'  
plt.show()
```



10) 플롯의 범례 설정 : legend()

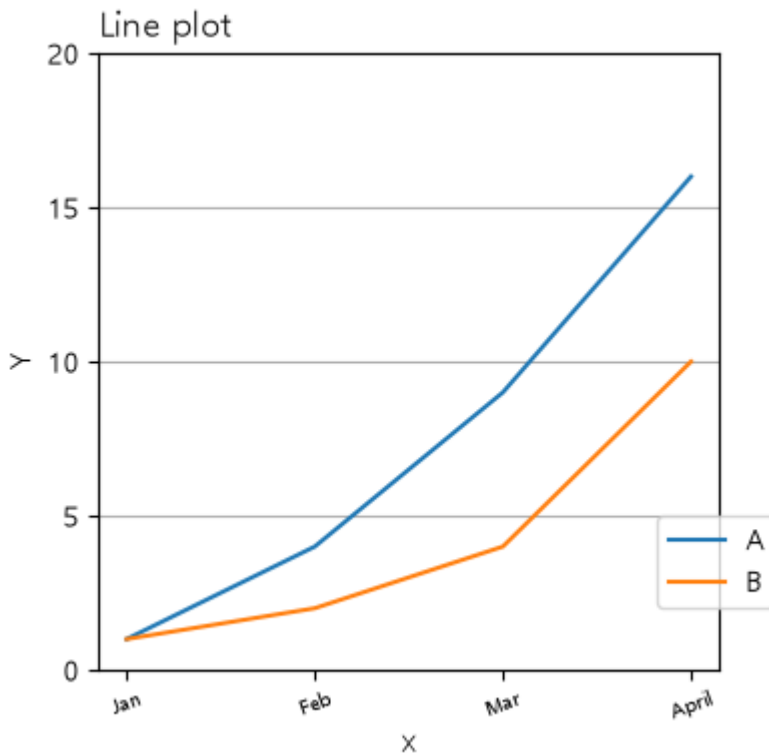
- plot에 label 속성이 추가되어 있어야 함
 - `plt.plot(x, y, label='a')`
- `plt.legend(loc=, ncol=)`
 - `loc = 0/1/2/3/4/5/6/7/8/9/10` # 범례표시 위치값
 - `loc = (x,y)`
 - `ncol=` 열갯수

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.legend.html

Location String	Location Code	설명
'best'	0	그래프의 최적의 위치에 표시합니다.(디폴트)
'upper right'	1	그래프의 오른쪽 위에 표시합니다.
'upper left'	2	그래프의 왼쪽 위에 표시합니다.
'lower left'	3	그래프의 왼쪽 아래에 표시합니다.
'lower right'	4	그래프의 오른쪽 아래에 표시합니다.
'right'	5	그래프의 오른쪽에 표시합니다.
'center left'	6	그래프의 왼쪽 가운데에 표시합니다.
'center right'	7	그래프의 오른쪽 가운데에 표시합니다.
'lower center'	8	그래프의 가운데 아래에 표시합니다.
'upper center'	9	그래프의 가운데 위에 표시합니다.
'center'	10	그래프의 가운데에 표시합니다.

```
In [43]: plt.figure(figsize=(4,4))
plt.plot([1,4,9,16], label='A')
```

```
plt.plot([1,2,4,10], label='B')
plt.title('Line plot', loc='left')
plt.ylim(0,20)
plt.xlabel('X', fontdict={'fontsize':8})
plt.ylabel('Y')
plt.xticks(ticks=[0,1,2,3], labels=['Jan', 'Feb', 'Mar', 'April'],
           rotation=20, fontdict={'fontsize':7})
plt.yticks([0,5,10,15,20])
plt.grid(axis='y') # axis='both', 'x', 'y'
# plt.legend(loc=2) # upper left
plt.legend(loc=(0.9,0.1))
# 그림영역의 세로 가로 범위가 (0,1)사이로 인식하므로 그 범위내에서 위치 지정
plt.show()
```



4. plot()

1) plot() 함수

- 기본으로 선을 그리는 함수
- 데이터가 시간, 순서 등에 따라 변화를 보여주기 위해 사용

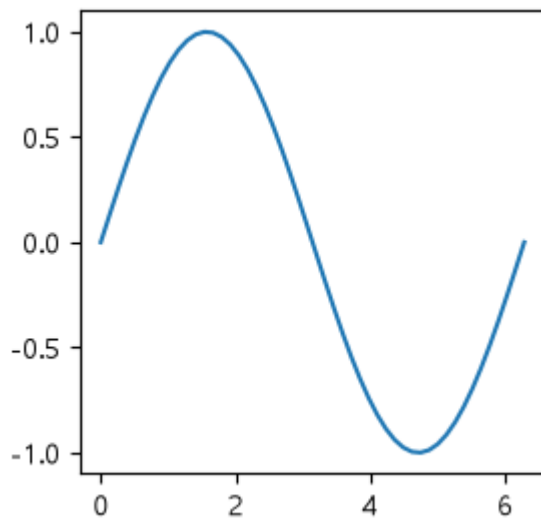
형식 :

matplotlib.pyplot.plot(*args, scalex=True, scaley=True, data=None, **kwargs)
 *args : x, y 축값을 리스트 형식으로 입력, y축값만 쓰는 경우 x축값은 자동 생성

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

```
In [46]: x = np.linspace(0, np.pi*2, 50)
y = np.sin(x)
plt.figure(figsize=(3,3))
```

```
plt.plot(x,y)
plt.show()
```

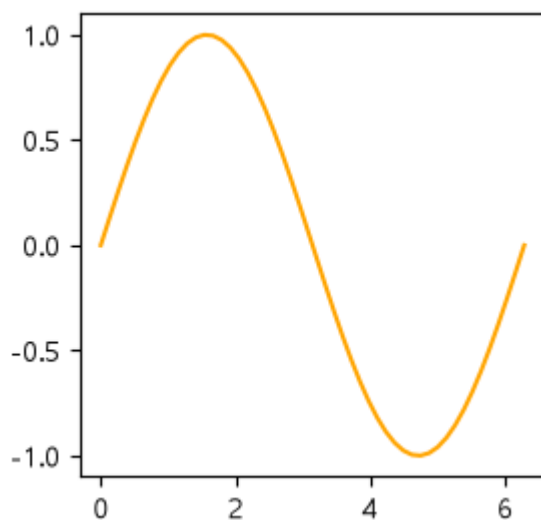


2) plot() 에서 사용되는 선 모양 관련 속성(약자로도 표기 가능)

- 선 색깔 : color (c)
- 선 굵기 : linewidth (lw)
- 선 스타일 : linestyle (ls)
- 마커 종류 : marker
- 마커 크기 : markersize (ms)
- 마커 선색깔 : markeredgecolor (mec)
- 마커 선굵기 : markeredgewidth (mew)
- 마커 내부색깔: markerfacecolor (mfc)

① 선 색 지정 : plt.plot(*args, color='색상명')

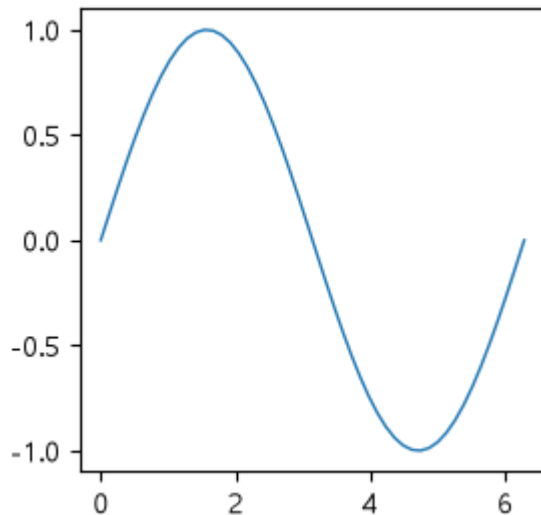
```
In [47]: x = np.linspace(0, np.pi*2, 50)
y = np.sin(x)
plt.figure(figsize=(3,3))
plt.plot(x,y, c='orange')
plt.show()
```



② 선 굵기 : plt.plot(*args, linewidth=정수값)

- linewidth 대신 lw 사용

```
In [49]: x = np.linspace(0, np.pi*2, 50)
y = np.sin(x)
plt.figure(figsize=(3,3))
plt.plot(x,y, lw=1) # linewidth=1
plt.show()
```



③ 선스타일(line style) 지정 : plt.plot(*args, linestyle=스타일이름)

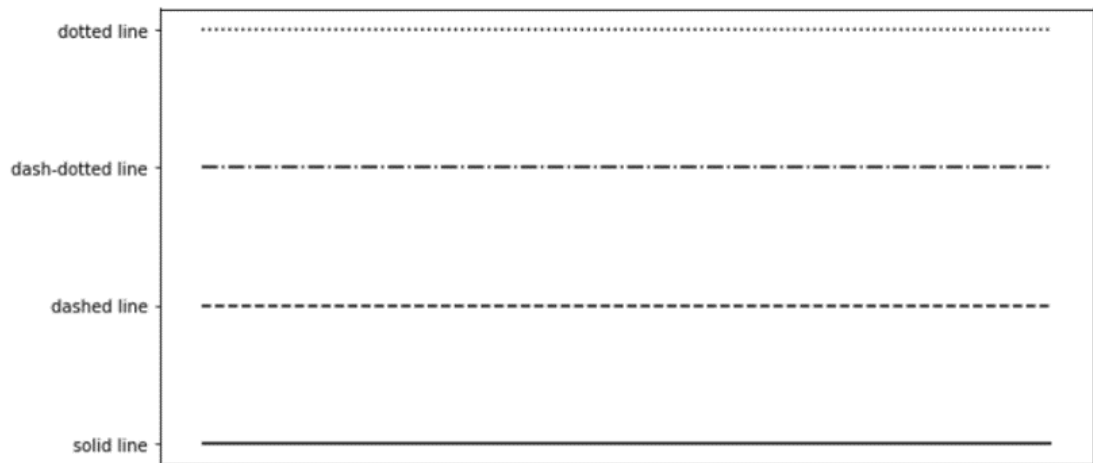
- linestyle = 'solid' | 'dotted' | 'dashed' | 'dashdot'

ls : {'-', '--', '-.', ':', '', (offset, on-off-seq), ...}

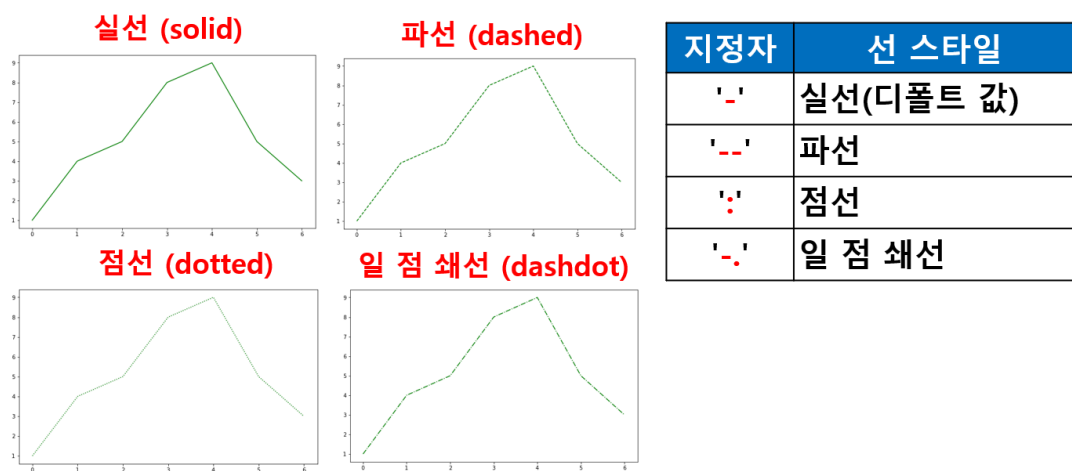
Possible values:

- A string:

linestyle	description
'-' or 'solid'	solid line
'--' or 'dashed'	dashed line
'-.' or 'dashdot'	dash-dotted line
':' or 'dotted'	dotted line
'none', 'None', '', or ''	draw nothing

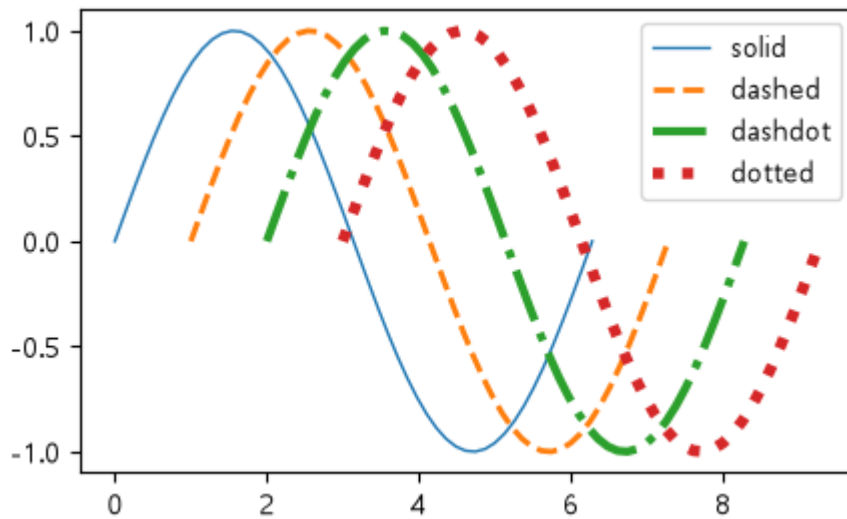


- 선스타일 기호



https://matplotlib.org/stable/api/_as_gen/matplotlib.lines.Line2D.html#matplotlib.lines.Line2D

```
In [50]: x = np.linspace(0, np.pi*2, 50)
y = np.sin(x)
plt.figure(figsize=(5,3))
for i, ls in enumerate(['solid', 'dashed', 'dashdot', 'dotted']):
    plt.plot(x+i, y, lw=i+1, ls=ls, label=ls) # linewidth=1, # linestyle
plt.legend()
plt.show()
```



④ 마커 지정 : plt.plot(*args, marker=마커유형)

- marker =

지정자	마커 유형
'+'	플러스 기호
'o'	원
'*'	별표
'.'	점
'x'	십자
's'	정사각형
'd'	다이아몬드
'^'	위쪽 방향 삼각형
'v'	아래쪽 방향 삼각형
'>'	오른쪽 방향 삼각형
'<'	왼쪽 방향 삼각형
'p'	오각별(펜타그램)
'h'	8각별(헥사그램)

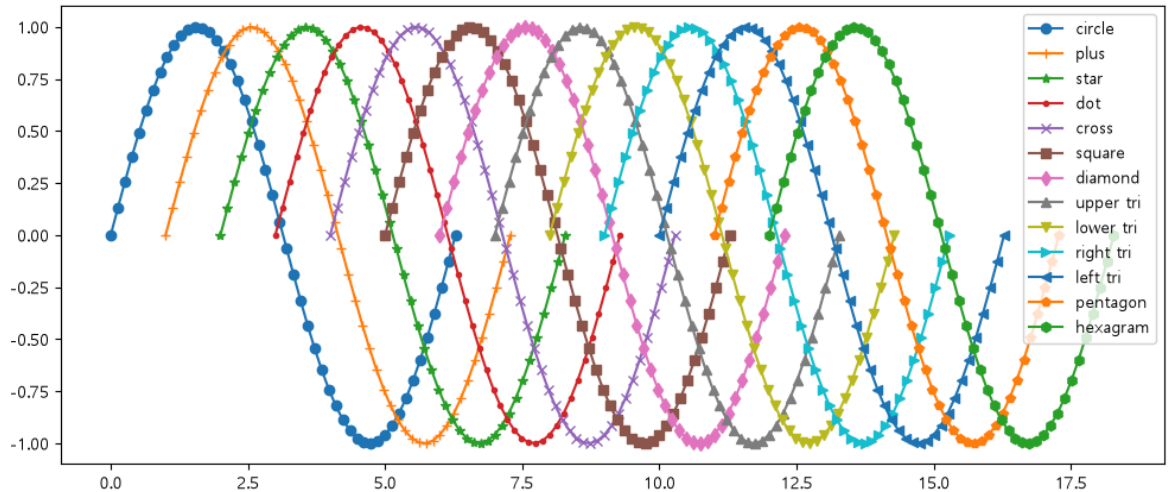
```
In [56]: markers = {'circle':'o', 'plus':'+', 'star':'*',
                    'dot': '.', 'cross':'x', 'square':'s',
                    'diamond':'d', 'upper tri':'^', 'lower tri':'v',
                    'right tri':'>', 'left tri':'<', 'pentagon':'p', 'hexagram':'h'}

x = np.linspace(0, np.pi*2, 50)
```

```

y = np.sin(x)
plt.figure(figsize=(12,5))
for i, item in enumerate(markers.items()):
    label = item[0]
    m = item[1]
    plt.plot(x+i, y, marker=m, label=label)
plt.legend()
plt.show()

```



⑤ 마커 크기 변경 : plt.plot(*args, markersize=)

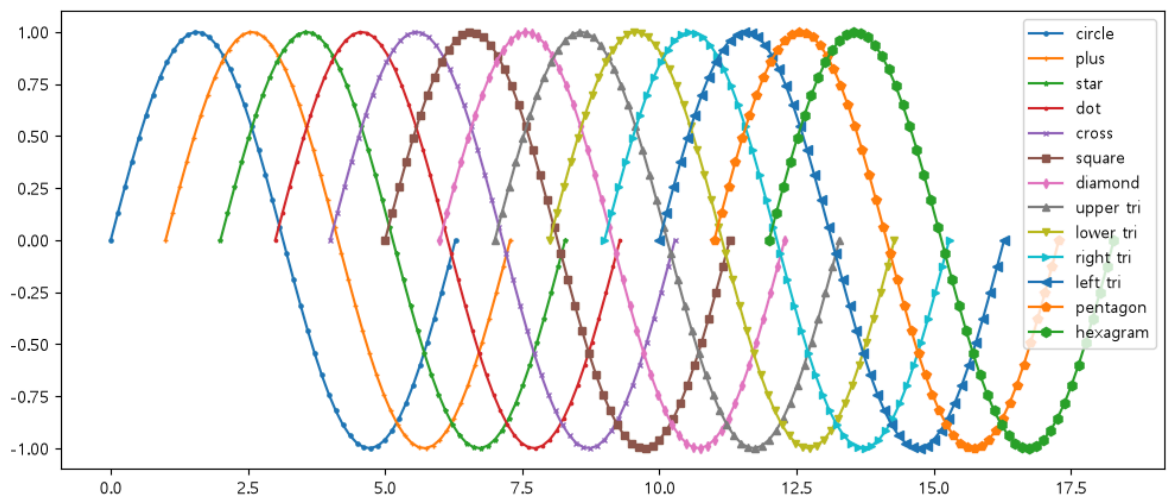
- 약어 : ms 사용

```

In [59]: markers = {'circle':'o', 'plus':'+', 'star':'*',
                    'dot':'.', 'cross':'x', 'square':'s',
                    'diamond':'d', 'upper tri':'^', 'lower tri':'v',
                    'right tri':'>', 'left tri':'<', 'pentagon':'p', 'hexagram':'h'}

x = np.linspace(0, np.pi*2, 50)
y = np.sin(x)
plt.figure(figsize=(12,5))
for i, item in enumerate(markers.items()):
    label = item[0]
    m = item[1]
    plt.plot(x+i, y, marker=m, label=label, markersize=(i+1)*0.3+2)
plt.legend()
plt.show()

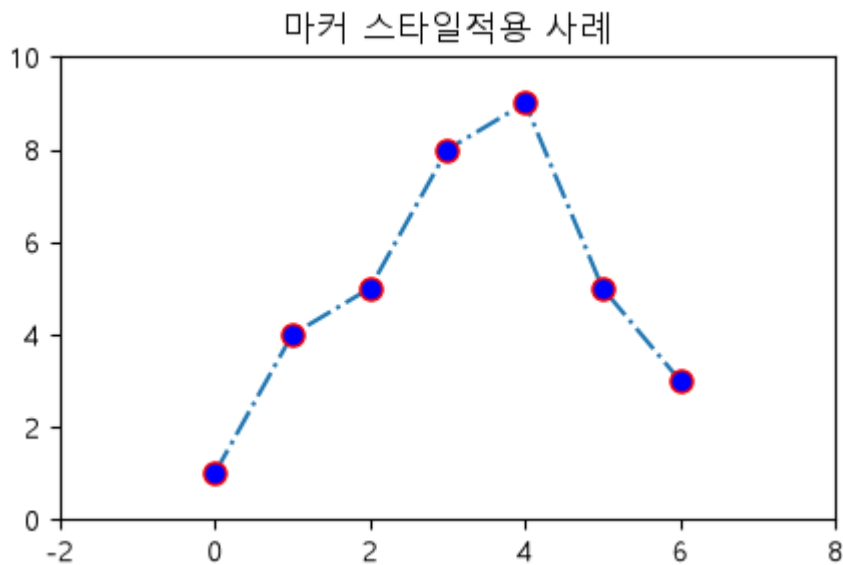
```



⑥ 마커 선굵기, 색상 변경

- 마커 선색깔 : `markeredgcolor (mec)`
- 마커 선굵기 : `markeredgewidth (mew)`
- 마커 내부색깔: `markerfacecolor (mfc)`

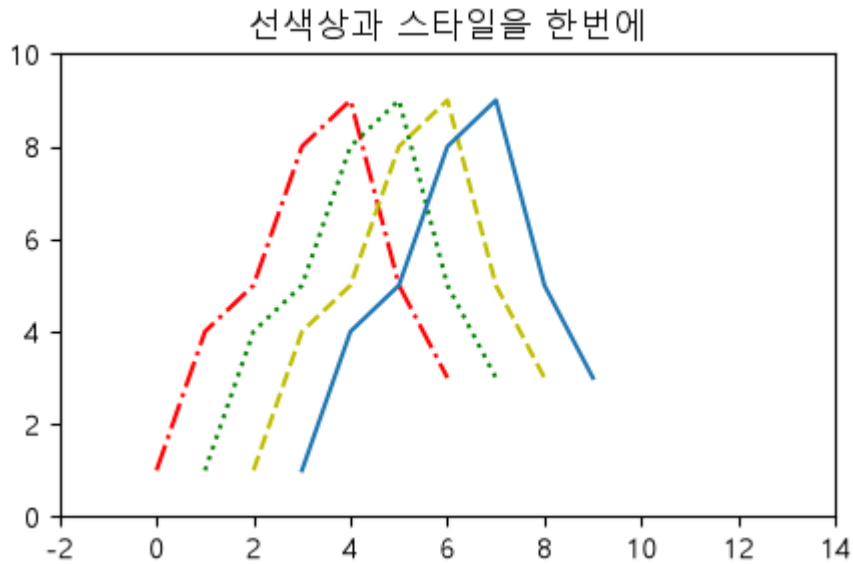
```
In [60]: y = [1,4,5,8,9,5,3]
plt.figure(figsize=(5,3))
plt.plot(y, ls='-.', marker='o', ms=8, mfc='blue', mec='r', mew=1) # markedg
plt.xlim(-2,8)
plt.ylim(0,10)
plt.title('마커 스타일적용 사례')
plt.show()
```



⑦ 선의 색상과 스타일을 한번에 지정하기

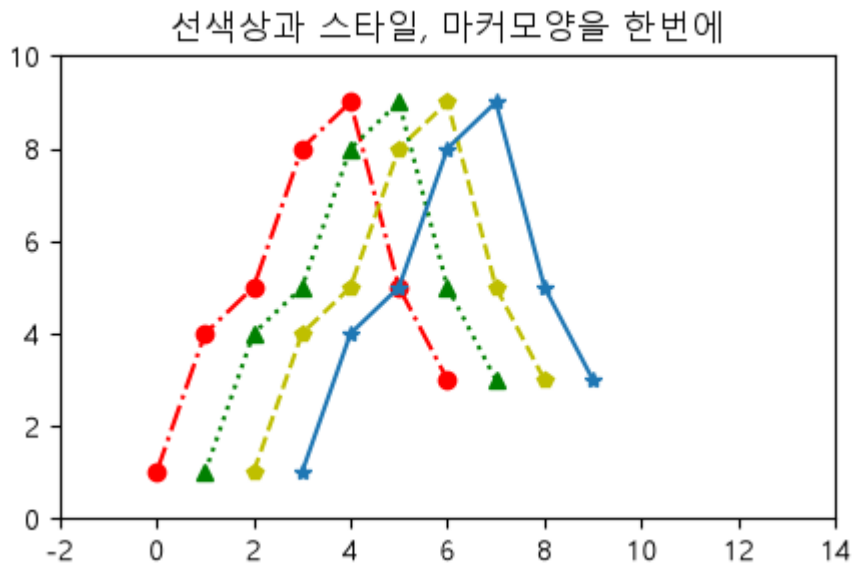
- 선모양 선색상에 해당하는 기호나 문자열을 축약하여 한 줄에 씀
- 예. 'r-', 'b--'

```
In [61]: y = [1,4,5,8,9,5,3]
x = np.arange(0, len(y))
plt.figure(figsize=(5,3))
plt.plot(x, y, 'r-.') # r(red), -. : dashdot
plt.plot(x+1, y, 'g:') # g(greed), : dotted
plt.plot(x+2, y, '--y') # y(yellow), -- : dashed
plt.plot(x+3, y, '-') # 기본색상, - : solid
plt.xlim(-2,14)
plt.ylim(0,10)
plt.title('선색상과 스타일을 한번에')
plt.show()
```



⑧ 선의 색상과 스타일, 마커모양을 한번에 지정하기

```
In [62]: y = [1,4,5,8,9,5,3]
x = np.arange(0, len(y))
plt.figure(figsize=(5,3))
plt.plot(x, y, 'r-.o') # r(red), -. : dashdot, o : circle
plt.plot(x+1, y, 'g:^') # g(green), : dotted, ^ : upper tri
plt.plot(x+2, y, '--yp') # y(yellow), -- : dashed, p: pentagon
plt.plot(x+3, y, '-*') # 기본색상, - : solid, *:star
plt.xlim(-2,14)
plt.ylim(0,10)
plt.title('선색상과 스타일, 마커모양을 한번에')
plt.show()
```



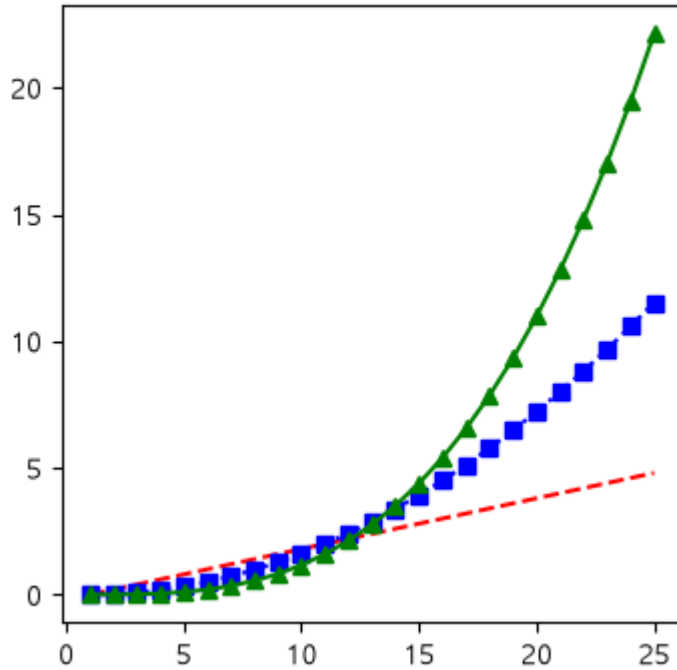
5. 여러 개의 플롯을 동일한 그림영역 내에 그리기

- plot()을 선그래프 수만큼 지정하거나, 하나의 plot() 함수 내에 지정하여 호출

```
In [63]: # 데이터
x = np.arange(1,26)
y = np.arange(0,5,0.2)
```

```
In [64]: # 여러 개 선그래프 그리기
plt.figure(figsize=(4,4))
plt.title('동일한 그림 영역에 여러 개의 선그래프 그리기')
plt.plot(x, y, 'r--')
plt.plot(x, 0.5*y**2, 'bs:')
plt.plot(x, 0.2*y**3, 'g^-')
plt.show()
```

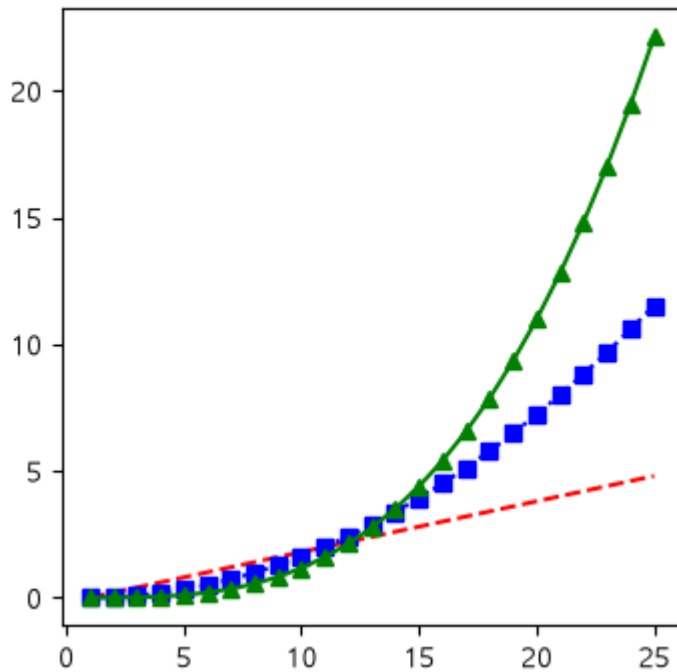
동일한 그림 영역에 여러 개의 선그래프 그리기



위 그래프 코드를 plot() 하나로 한번에 표현하기

```
In [65]: plt.figure(figsize=(4,4))
plt.title('동일한 그림 영역에 여러 개의 선그래프 그리기')
plt.plot(x, y, 'r--', x, 0.5*y**2, 'bs:', x, 0.2*y**3, 'g^-')
plt.show()
```

동일한 그림 영역에 여러 개의 선그래프 그리기



6. 하나의 그림 영역을 분할하여 여러 플롯 표시

1) plt.subplot() 함수

하나의 윈도우(figure)안에 여러 개의 플롯을 배열 형태로 표시

- 그리드 형태의 Axes 객체들 생성
- 형식 : subplot(인수1, 인수2, 인수3)
 - 인수1 과 인수2는 전체 그리드 행렬 모양 지시
 - 인수3 : 그래프 위치 번호
 - subplot(2,2,1) 가 원칙이나 줄여서 221로 쓸 수 있음
 - subplot(221) 2행 2열의 그리드에서 첫번째 위치
 - subplot(224) 2행 2열의 그리드에서 마지막 위치
- subplot()은 show()하기 전에 먼저 위치 설정해야 함

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplot.html

- tight_layout(pad=) : 플롯간 간격을 설정
 - pad = 간격값(실수)

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.tight_layout.html

2 x 2 형태의 네 개의 플롯

```
In [66]: np.random.seed(0)
plt.figure(figsize=(6,5))

plt.subplot(221)
plt.plot(np.random.rand(5))
```

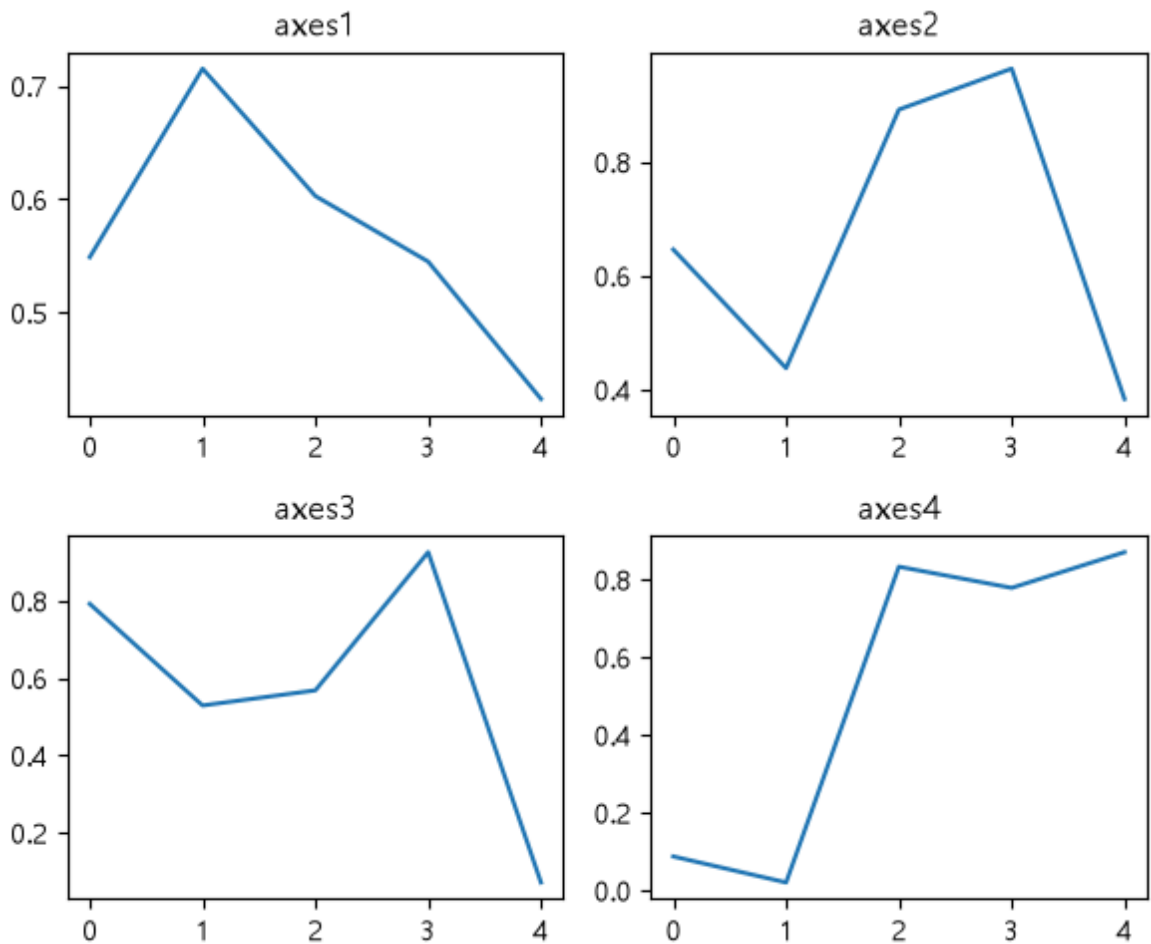
```
plt.title('axes1')

plt.subplot(222)
plt.plot(np.random.rand(5))
plt.title('axes2')

plt.subplot(223)
plt.plot(np.random.rand(5))
plt.title('axes3')

plt.subplot(224)
plt.plot(np.random.rand(5))
plt.title('axes4')

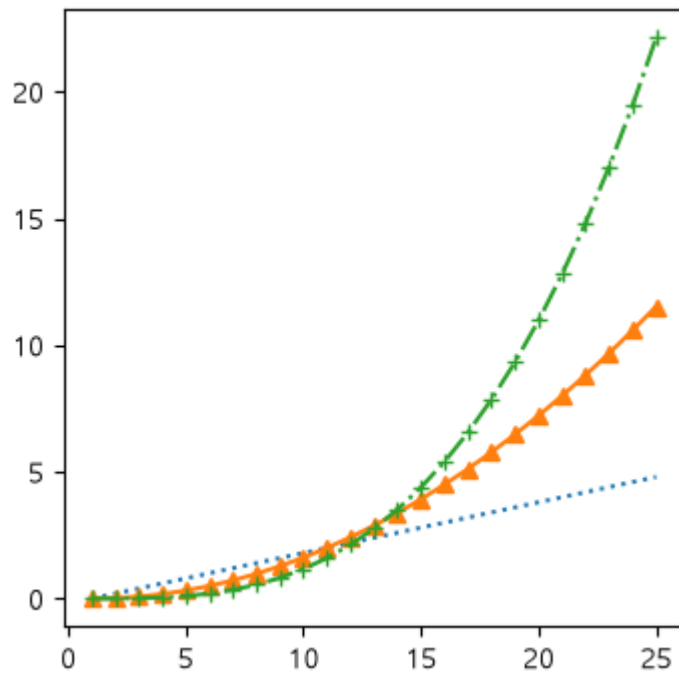
plt.tight_layout()
plt.show()
```



문제1. 다음 코드에 제시한 데이터를 이용하여 세 플롯을 subplot() 이용하여 그리기

```
In [67]: x = np.arange(1,26)
y = np.arange(0, 5, 0.2)
y2 = 0.5*y**2
y3 = 0.2*y**3

plt.figure(figsize=(4,4))
plt.plot(x, y, ':')
plt.plot(x, y2, '^-.')
plt.plot(x, y3, '+-.')
plt.show()
```



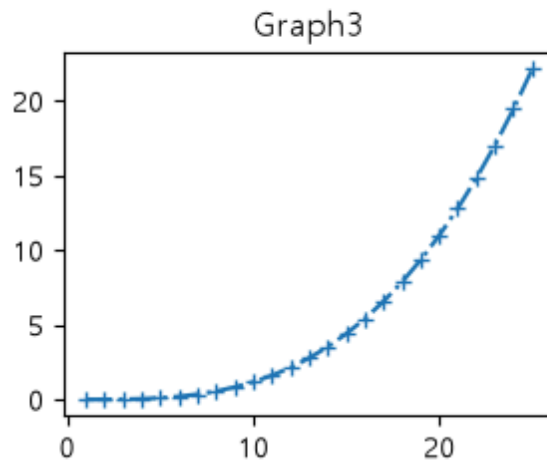
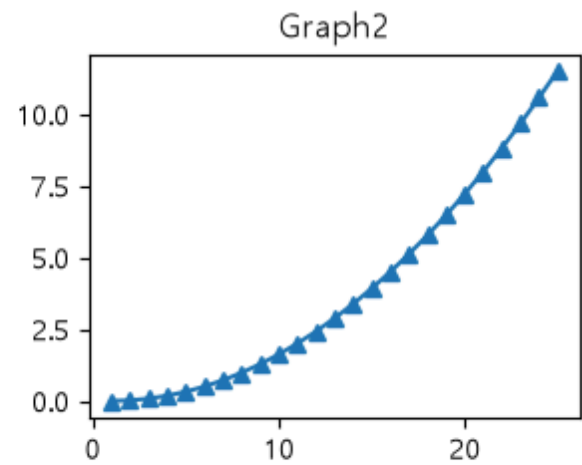
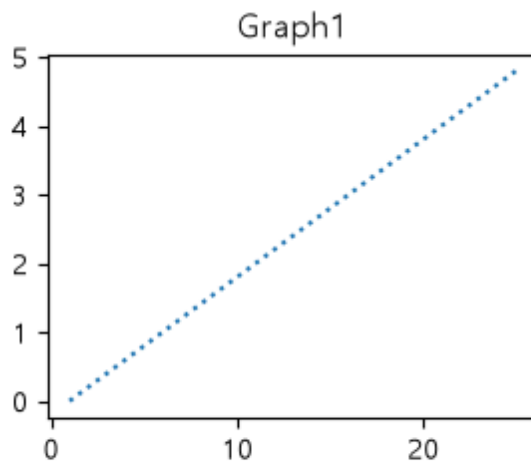
```
In [68]: plt.figure(figsize=(6,5))
```

```
plt.subplot(221)
plt.plot(x, y, ':.')
plt.title('Graph1')
```

```
plt.subplot(222)
plt.plot(x, y2, '^-.')
plt.title('Graph2')
```

```
plt.subplot(223)
plt.plot(x, y3, '+-.')
```

```
plt.tight_layout()
plt.show()
```



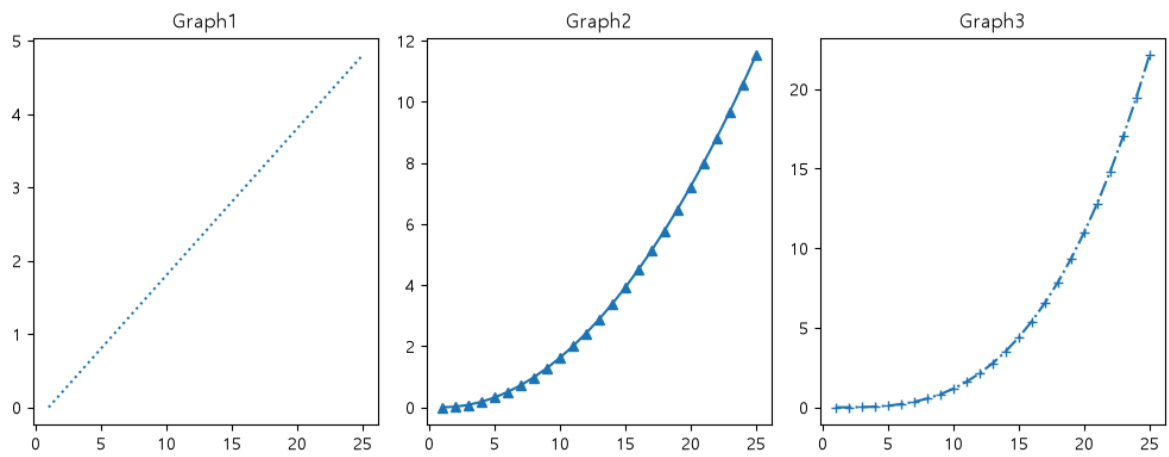
```
In [71]: plt.figure(figsize=(10, 4))
```

```
plt.subplot(1,3,1)  
plt.plot(x, y, ':')  
plt.title('Graph1')
```

```
plt.subplot(1,3,2)  
plt.plot(x, y2, '^-')  
plt.title('Graph2')
```

```
plt.subplot(1,3,3)  
plt.plot(x, y3, '+-.')  
plt.title('Graph3')
```

```
plt.tight_layout()  
plt.show()
```



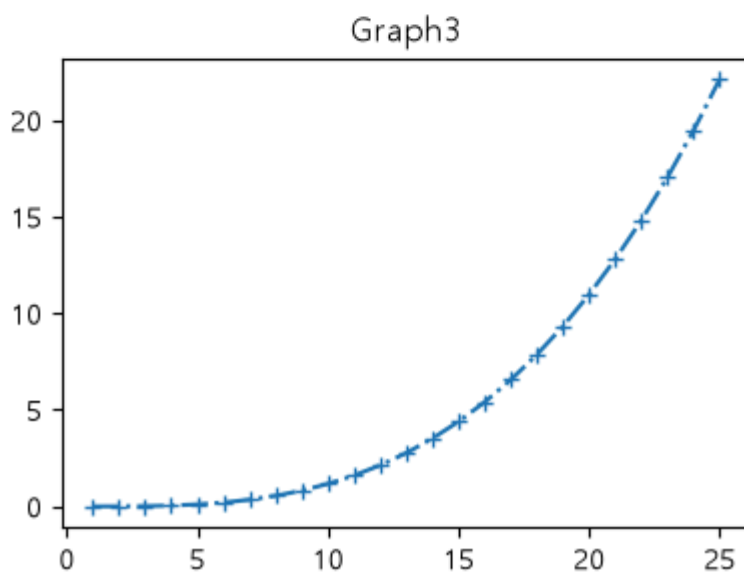
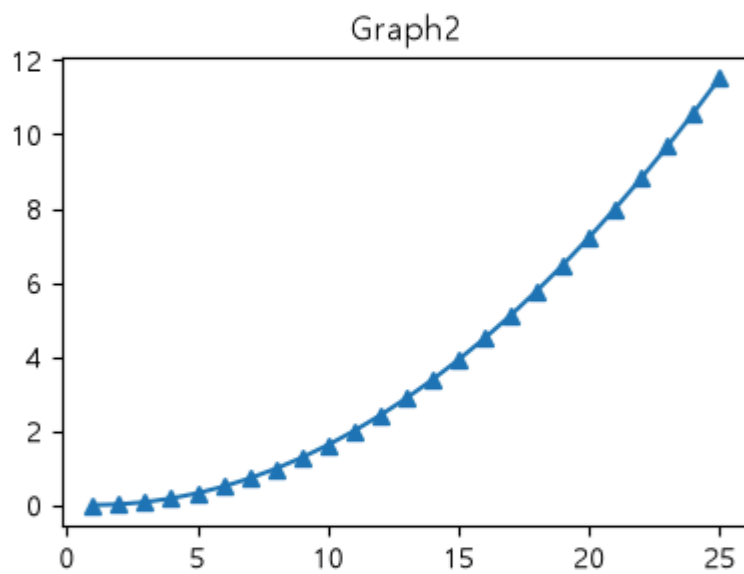
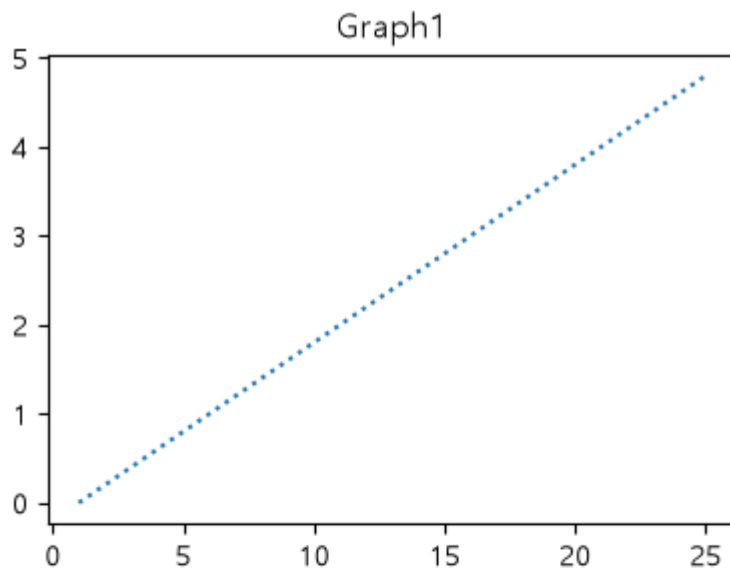
In [75]: `plt.figure(figsize=(4,9))`

```
plt.subplot(311)
plt.plot(x, y, ':')
plt.title('Graph1')

plt.subplot(312)
plt.plot(x, y2, '^-.')
plt.title('Graph2')

plt.subplot(313)
plt.plot(x, y3, '+-.')
plt.title('Graph3')

plt.tight_layout()
plt.show()
```



2) plt.subplots(행,열) 함수

여러 개의 Axes 객체를 동시에 생성해주는 함수

- 행렬 형태의 객체로 반환

- 두 개의 반환값이 있음
 - 첫 번째 반환은 그래프 객체 전체 이름 : 거의 사용하지 않음
 - 두 번째 반환값에 Axes 객체를 반환함
 - 두 번째 반환값이 필요하므로 반환 값 모두를 반환받아 두 번째 값을 사용해야 함
- ex. fig, axes = plt.subplots(2,2)

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html

```
In [76]: fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(8,8))

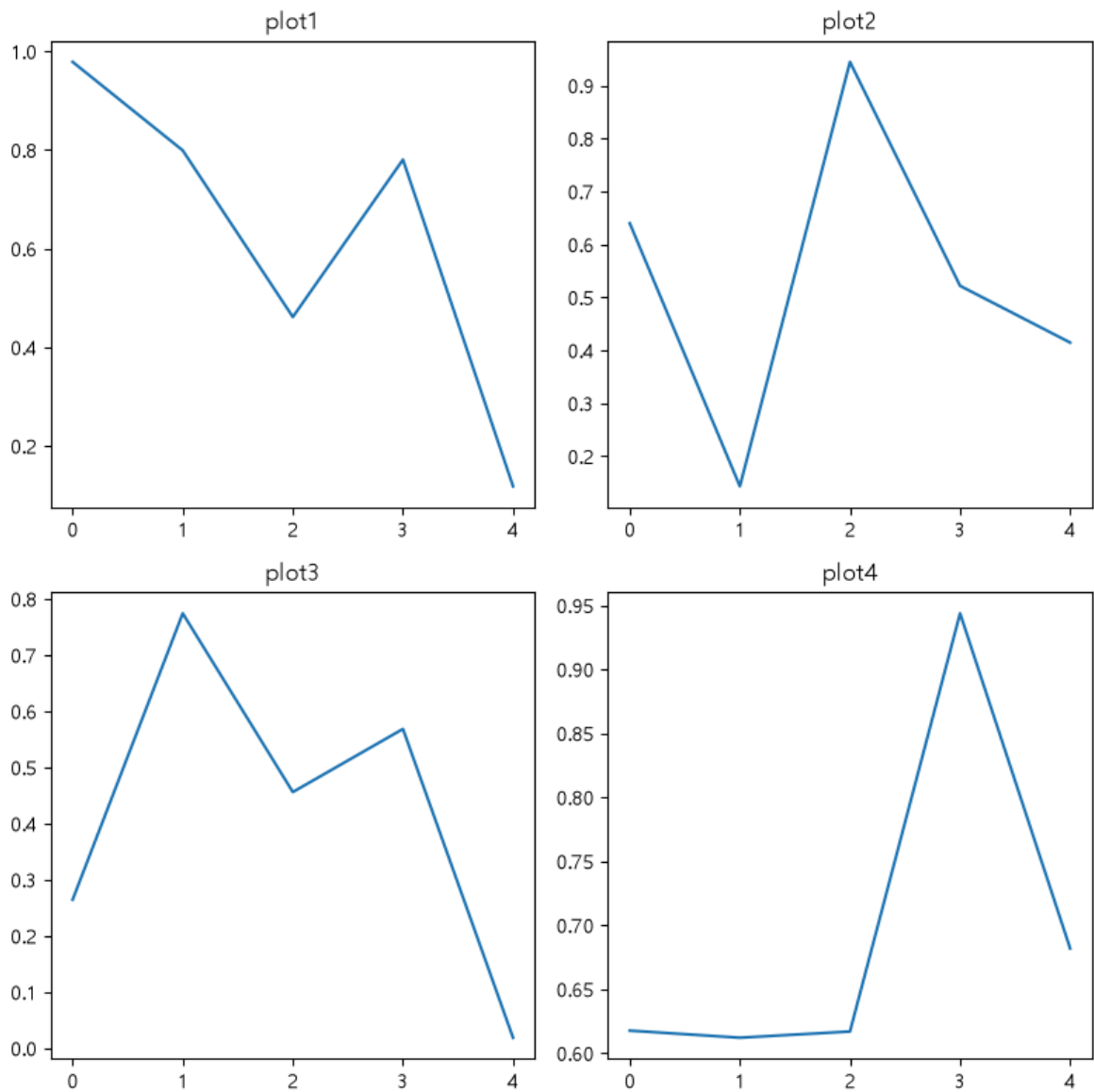
axes[0,0].plot(np.random.rand(5))
axes[0,0].set_title('plot1')

axes[0,1].plot(np.random.rand(5))
axes[0,1].set_title('plot2')

axes[1,0].plot(np.random.rand(5))
axes[1,0].set_title('plot3')

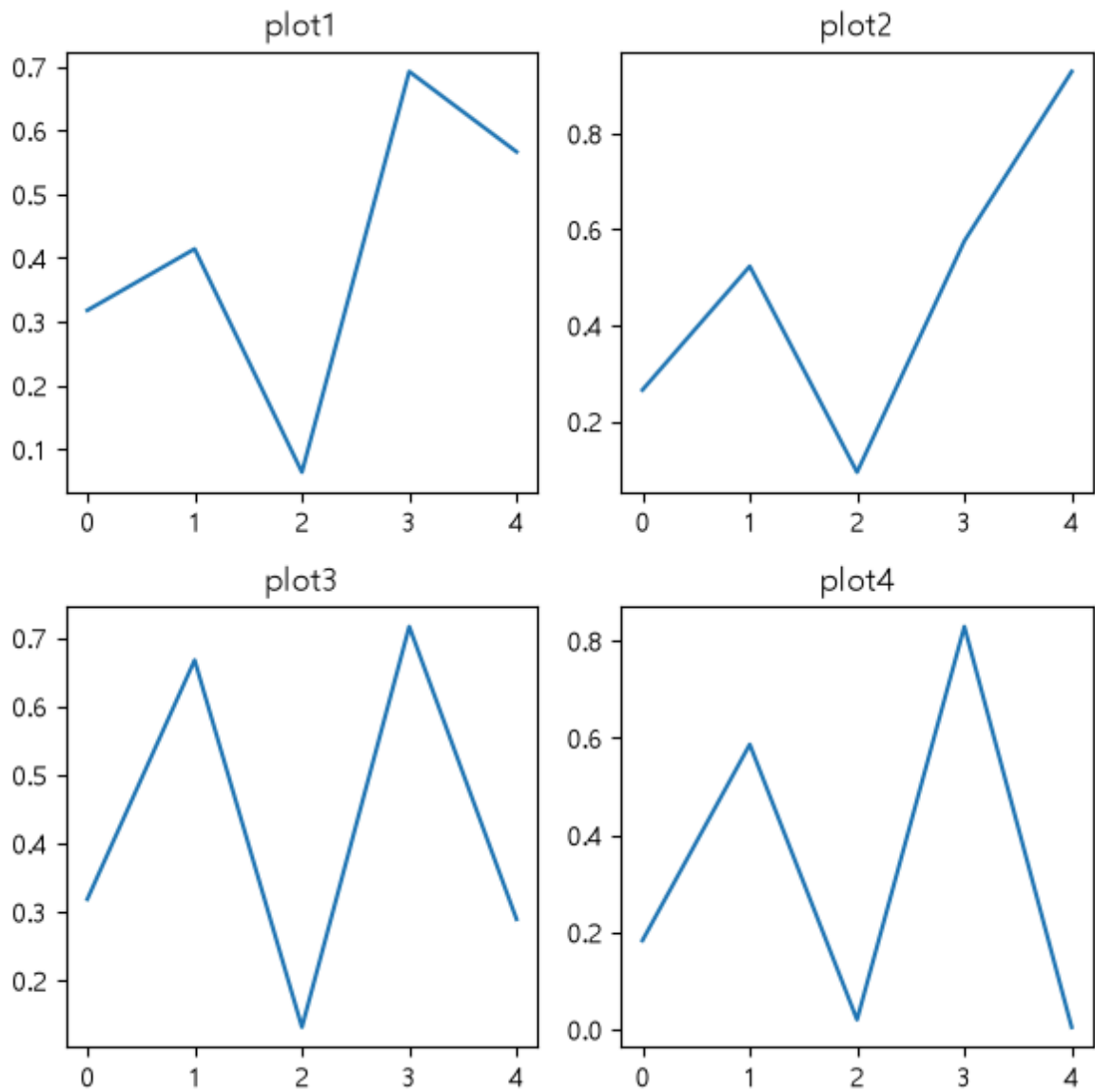
axes[1,1].plot(np.random.rand(5))
axes[1,1].set_title('plot4')

plt.tight_layout()
plt.show()
```



```
In [80]: nrows, ncols = 2,2
fig, axes = plt.subplots(nrows=nrows, ncols=ncols,
                          figsize=(6,6), tight_layout=True)

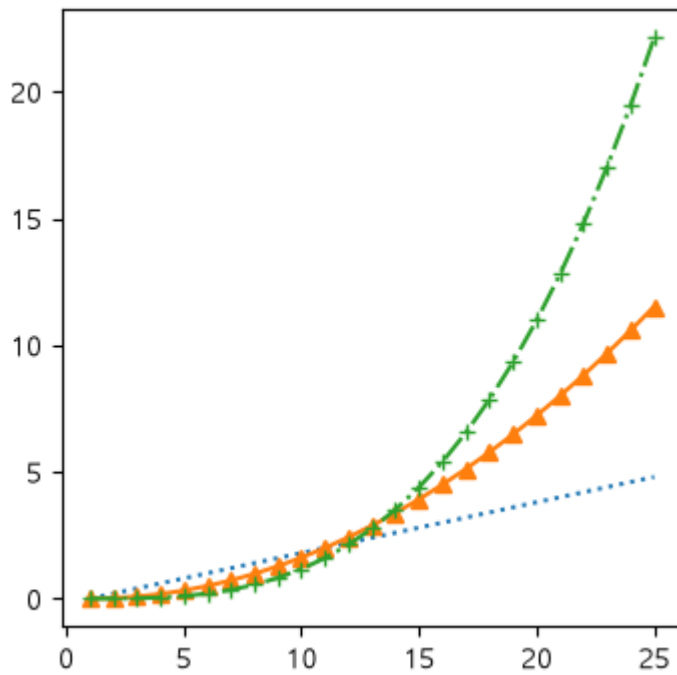
# for i in range(0, nrows):
#     for j in range(0, ncols):
#         axes[i,j].plot(np.random.rand(5))
#         axes[i,j].set_title(f'plot{ncols*i+j+1}')
n_fig = 4
for n in range(n_fig):
    i, j = n//ncols, n%ncols
    axes[i,j].plot(np.random.rand(5))
    axes[i,j].set_title(f'plot{n+1}')
plt.show()
```



문제2. 다음 코드에 제시한 데이터를 이용하여 세 플롯을 subplots() 이용하여 그리기

```
In [81]: x = np.arange(1,26)
y = np.arange(0, 5, 0.2)
y2 = 0.5*y**2
y3 = 0.2*y**3

plt.figure(figsize=(4,4))
plt.plot(x, y, ':')
plt.plot(x, y2, '^-.')
plt.plot(x, y3, '+-.')
plt.show()
```

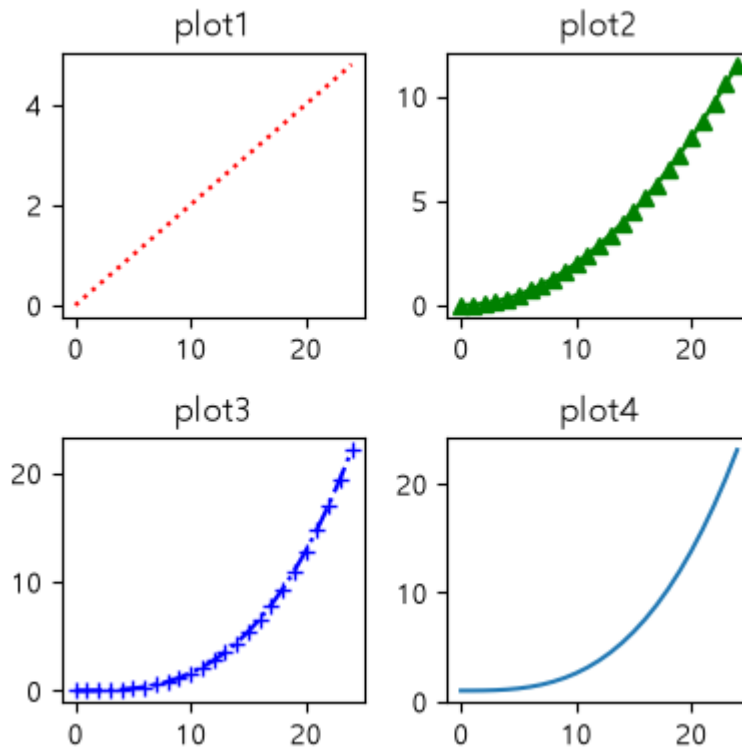


```
In [97]: # plot의 수에 따라서 그리기
x = np.arange(1,26)
y = np.arange(0, 5, 0.2)
y2 = 0.5*y**2
y3 = 0.2*y**3
data_y = [y, y2, y3, y3+1]
style = ['r:', 'g^-', 'b+-.', '-']

nrows, ncols = 2,2
n_fig = nrows*ncols
fig, axes = plt.subplots(nrows=nrows, ncols=ncols,
                        figsize=(2*ncols,2*nrows), tight_layout=True)

if ncols==1 or nrows==1:
    for n in range(n_fig):
        axes[n].plot(data_y[n], style[n])
        axes[n].set_title(f'plot{n+1}')
else:
    for n in range(n_fig):
        i,j = n//ncols, n%ncols
        axes[i][j].plot(data_y[n], style[n])
        axes[i][j].set_title(f'plot{n+1}')

plt.show()
```

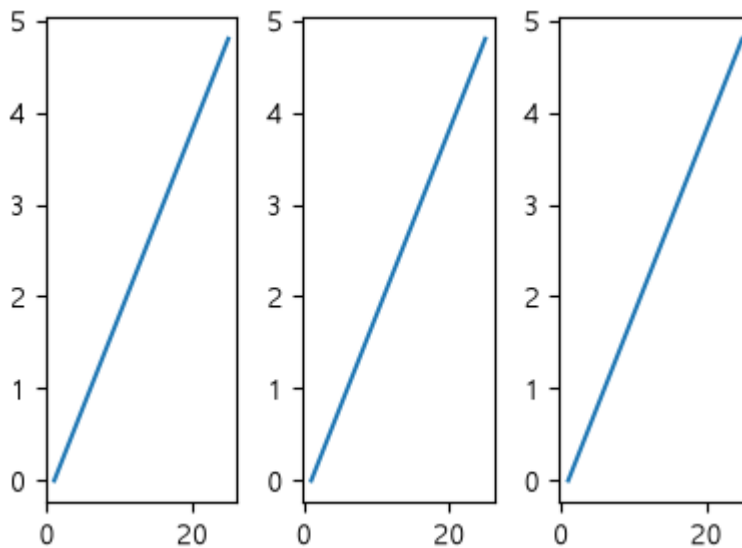


In [103...

```
nrows, ncols = 1,3
fig,(axs1, axs2, axs3) = plt.subplots(nrows=nrows, ncols=ncols,
                                       figsize=(4,3), tight_layout=True)

axs1.plot(x,y)
axs2.plot(x,y)
axs3.plot(x,y)

plt.show()
```



3) plt.subplot_mosaic() 함수

- 서로 다른 크기의 영역으로 구분하여 플롯 그리기

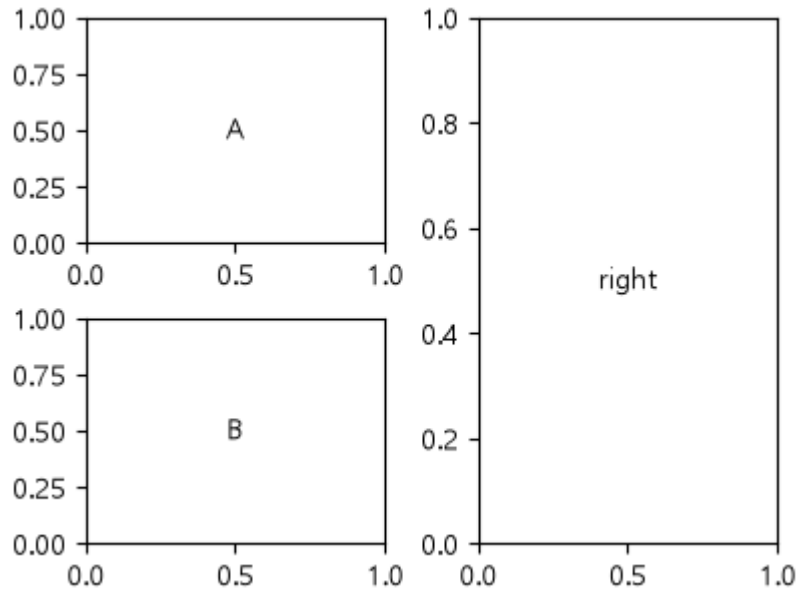
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplot_mosaic.html

In [105...

```
fig, axs = plt.subplot_mosaic([[ 'A', 'right'], [ 'B', 'right']],
                              figsize=(4,3),
                              layout='constrained')

print(axs)
for ax_name in axs:
    axs[ax_name].text(0.5, 0.5, ax_name, ha='center', va='center')
```

{'A': <Axes: label='A'>, 'right': <Axes: label='right'>, 'B': <Axes: label='B'>}



In [106...

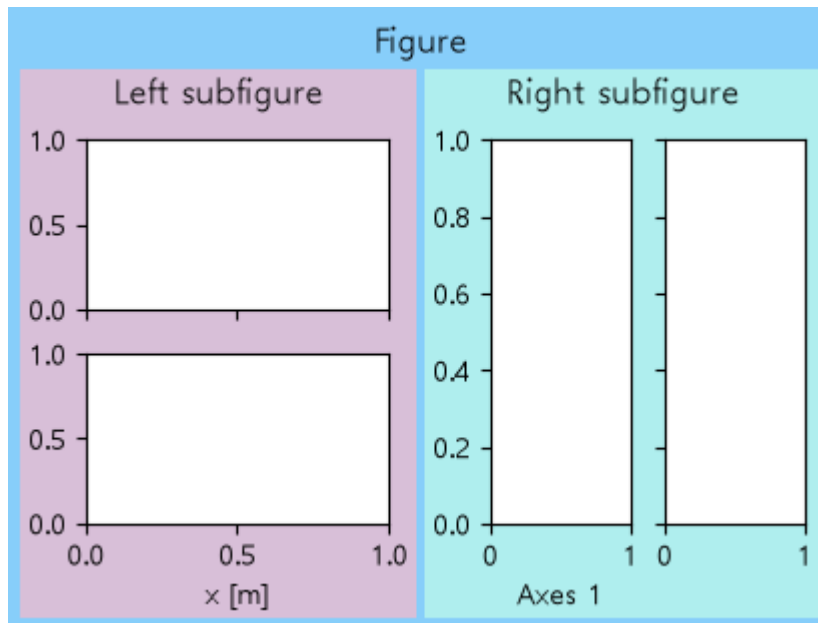
```
fig = plt.figure(figsize=(4,3), layout='constrained', facecolor='lightskyblue')
fig.suptitle('Figure')
figL, figR = fig.subfigures(1,2)

figL.set_facecolor('thistle')
axL = figL.subplots(2,1, sharex=True)
axL[1].set_xlabel('x [m]')
figL.suptitle('Left subfigure')

figR.set_facecolor('paleturquoise')
axR = figR.subplots(1,2, sharey=True)
axR[0].set_xlabel('Axes 1')
figR.suptitle('Right subfigure')
```

Out[106...

Text(0.5, 0.98, 'Right subfigure')



subplot(), subplots(), subplot_mosaic() 비교

	subplot()	subplots()	subplot_mosaic()
코드 방식	subplot을 하나씩 수동 지정	subplot을 배열로 일괄 생성	문자열/딕셔너리 기반으로 유연하게 레이아웃 구성
접근 방식	번호로 직접 접근 (plt.subplot(1,2,1))	배열의 인덱싱 (axes[0,1], axes[1], axes[0][1])	이름(label)으로 접근(ax['A'],ax['B'])
반환 값	단일 Axes Subplot 객체	(Figure, Axes) 튜플 (Axes 배열 또는 객체 배열)	(Figure, dict[str, Axes]) 형태로 반환
코드 구조	반복문 없이 단순한 코드지만 중복 많음	반복문 활용 가능. 배열 인덱싱 필요	레이아웃 정의가 직관적이고 가독성 좋음. 반복문도 가능
레이아웃 유연성	낮음	중간(격자형만 가능)	매우 높음(불균형, 중첩, 특정 영역 확대 등 자유롭게 설정 가능)
권장 사용	아주 단순한 1xN 또는 N행의 그래프	다수 subplot을 일정한 행렬 형태로 반복하여 그릴 때	복잡한 레이아웃, 대시보드 구성, 이름 기반으로 명확히 구분할 때
버전 요구 사항	오래된 버전부터 사용 가능	오래된 버전부터 사용 가능	Matplotlib 3.4 이상
예시 코드	plt.subplot(2,1,1)	fig, axes = plt.subplots(2,2)	fig, axes = plt.subplot_mosaic([['A','B'], ['C','B']]))
요약	subplot 하나씩 단순하게 붙이기	반복적인 플롯 그리기	이름으로 구역 구분, 복잡한 배치

지원 되는 플롯 유형